LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Ashma Salimah NIM. 2410817220015

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Ashma Salimah
NIM : 2410817220015

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 0
DAFTAR ISI 1
DAFTAR GAMBAR 2
DAFTAR TABEL 3
SOAL 1 4
A. Source Code 6
B. Output Program 14
C. Pembahasan 17
SOAL 2 29
D. Pembahasan 29
SOAL ESSAY 33
A. Pembahasan 33
TAUTAN GITHUB 38
DAFTAR PUSTAKA 39

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 4
Gambar 2. Tampilan Pilihan Persentase Tip 5
Gambar 3. Tampilan Aplikasi Setelah Dijalankan 5
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal 14
Gambar 5. Screenshot Hasil Jawaban Soal XML Persentase Tip 14
Gambar 6. Screenshot Hasil Jawaban Soal XML Setelah Dijalankan 15
Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal 15
Gambar 8. Screenshot Hasil Jawaban Soal Compose Persentase Tip 16
Gambar 9. Screenshot Hasil Jawaban Soal Compose Setelah Dijalankan 16

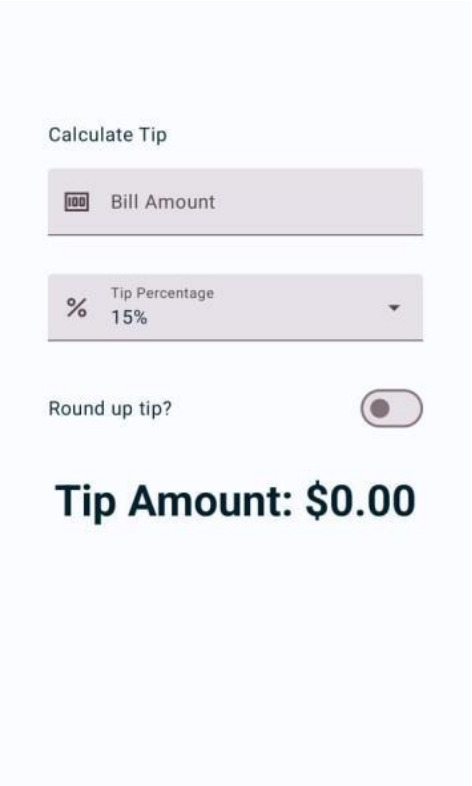
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML 6
Tabel 2. Source Code MainActivity.kt Compose 7
Tabel 3. Source Code activity_main.xml 10

SOAL 1

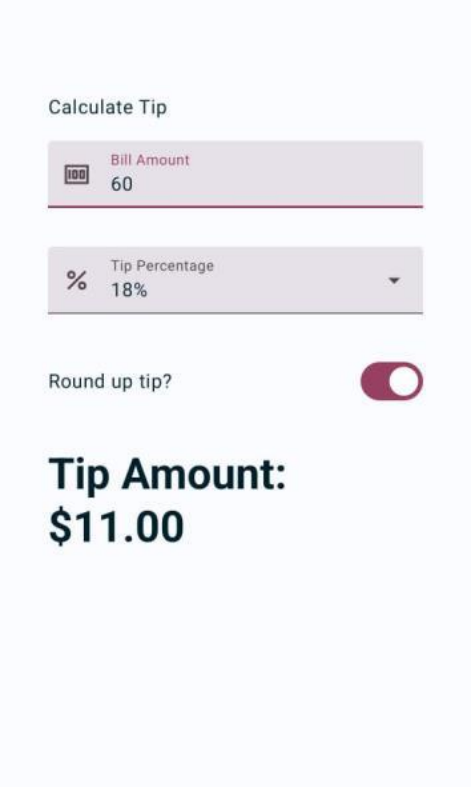
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi

Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

Tabel 1. Source Code MainActivity.kt XML

No	MainActivity.kt (XML)
1	package com.example.xml
2	
3	import android.os.Bundle
4	import android.widget.ArrayAdapter
5	import androidx.core.widget.addTextChangedListener
6	import androidx.appcompat.app.AppCompatActivity
7	import com.example.xml.databinding.ActivityMainBinding
8	
9	class MainActivity : AppCompatActivity() {
10	private lateinit var binding: ActivityMainBinding
11	
12	override fun onCreate(savedInstanceState: Bundle?) {
13	super.onCreate(savedInstanceState)
14	binding = ActivityMainBinding.inflate(*layoutInflater*)
15	setContentView(binding.*root*)
16	
17	val tipOption = *listOf*("15%", "18%", "20%")
18	val adapter = ArrayAdapter(this, android.R.layout.*simple_dropdown_item_1line*, tipOption)
19	binding.ACTipPercent.setAdapter(adapter)
20	binding.ACTipPercent.setText(tipOption[0], false)
21	
22	fun calculateTip(){
23	val billAmount = binding.TIEBill.*text*.*toString*().*toDoubleOrNull*() ?: 0.0
24	val tipPercentage = when (binding.ACTipPercent.*text*.toString()){
25	"15%" -> 0.15
26	"18%" -> 0.18
27	"20%" -> 0.20
28	else -> 0.0
29	}
30	var tipAmount = billAmount * tipPercentage
31	if (binding.SRound.*isChecked*){
32	tipAmount = kotlin.math.*ceil*(tipAmount)
33	}
34	binding.TVTipAmount.*text* = "Tip Amount: ${"$"}${"%.2f".*format*(tipAmount)}"
35	}
36	

```
37 binding.TIEBill.addTextChangedListener{calculateTip()}
38         binding.ACTipPercent.setOnItemClickListener{_,
   _, _, _ -> calculateTip()}
39         binding.SRound.setOnCheckedChangeListener { _, _
   -> calculateTip()}
40     }
41 }
```

Tabel 2. Source Code MainActivity.kt Compose

No	Main Activity.kt (Compose)
1	package com.example.composeModul2
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.layout.*
8	import androidx.compose.foundation.text.KeyboardOptions
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.*
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.text.font.FontWeight
14	import androidx.compose.ui.text.input.KeyboardType
15	import androidx.compose.ui.unit.dp
16	import androidx.compose.ui.unit.sp
17	import com.example.composeModul2.ui.theme.COMPOSETheme
18	import kotlin.math.ceil
19	import androidx.compose.material.icons.Icons
20	import androidx.compose.material.icons.filled.Money
21	import androidx.compose.material.icons.filled.Percent
22	
23	
24	class MainActivity : ComponentActivity() {
25	override fun onCreate(savedInstanceState: Bundle?) {
26	super.onCreate(savedInstanceState)
27	enableEdgeToEdge()
28	setContent {
29	COMPOSETheme {
30	TipCalculatorApp()
31	}
32	}
33	}

```
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TipCalculatorApp() {
    var billAmount by remember { mutableStateOf("") }
    var selectedTip by remember { mutableStateOf("15%") }
    var roundUp by remember { mutableStateOf(false) }
    var expanded by remember { mutableStateOf(false) }

    val tipOptions = listOf("15%", "18%", "20%")

    val tipAmount = calculateTipAmount(billAmount, selectedTip, roundUp)

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(horizontal = 32.dp, vertical = 150.dp)
    ) {
        Text(
            text = "Calculate Tip",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold
        )

        Spacer(modifier = Modifier.height(16.dp))

        TextField(
            value = billAmount,
            onValueChange = { billAmount = it },
            label = { Text("Bill Amount") },
            leadingIcon = { Icon(imageVector = Icons.Default.Money, contentDescription = null) },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(16.dp))

        ExposedDropdownMenuBox(
            expanded = expanded,
            onExpandedChange = { expanded = !expanded
}
```

```
        ) {
            TextField(
                value = selectedTip,
                onValueChange = {},
                readOnly = true,
                label = { Text("Tip Percentage") },
                leadingIcon = { Icon(imageVector = Icons.Default.Percent, contentDescription = null) },
                trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
                modifier = Modifier
                    .fillMaxWidth()
                    .menuAnchor()
            )
            ExposedDropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false }
            ) {
                tipOptions.forEach { option ->
                    DropdownMenuItem(
                        text = { Text(option) },
                        onClick = {
                            selectedTip = option
                            expanded = false
                        }
                    )
                }
            }
        }

        Spacer(modifier = Modifier.height(16.dp))

        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(text = "Round up tip?", fontSize = 16.sp)
            Switch(
                checked = roundUp,
                onCheckedChange = { roundUp = it }
            )
```

```
115         }
116
117         Spacer(modifier = Modifier.height(32.dp))
118
119         Text(
120             text = "Tip Amount:
    $${"%.2f".format(tipAmount)}",
121             fontSize = 28.sp,
122             fontWeight = FontWeight.Bold
123         )
124     }
125 }
126
127 fun calculateTipAmount(billAmount: String,
    tipPercentage: String, roundUp: Boolean): Double {
128     val amount = billAmount.toDoubleOrNull() ?: 0.0
129     val percentage = when (tipPercentage) {
130         "15%" -> 0.15
131         "18%" -> 0.18
132         "20%" -> 0.20
133         else -> 0.15
134     }
135     var tip = amount * percentage
136     if (roundUp) tip = ceil(tip)
137     return tip
138 }
```

Tabel 3. Source Code activity_main.xml

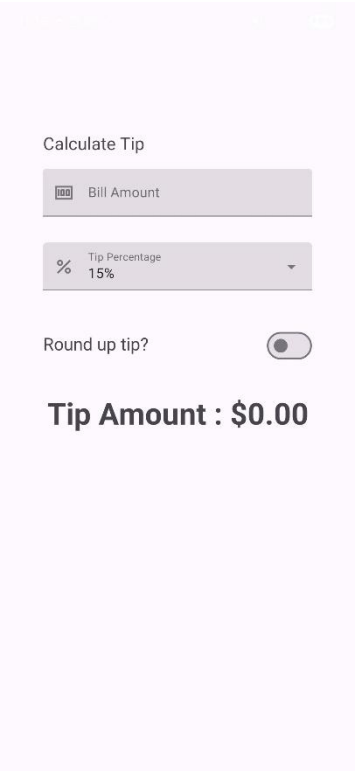
No	activity_main.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/andr oid"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/main"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	tools:context=".MainActivity">
10	
11	<TextView
12	android:id="@+id/TVCalculate"
13	android:layout_width="wrap_content"
14	android:layout_height="wrap_content"

```
        android:layout_marginStart="50dp"
        android:layout_marginTop="150dp"
        android:text="Calculate Tip"
        android:textSize="20sp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />


<com.google.android.material.textfield.TextInputLayout
        android:id="@+id/TIBill"
style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="50dp"
        android:layout_marginTop="16dp"
        android:layout_marginEnd="50dp"
        android:hint="Bill Amount"
app:startIconDrawable="@drawable/baseline_money_24"
app:layout_constraintTop_toBottomOf="@id/TVCalculate"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">


<com.google.android.material.textfield.TextInputEditText
            android:id="@+id/TIEBill"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:paddingStart="50dp"
            android:inputType="numberDecimal" />
</com.google.android.material.textfield.TextInputLayout>


<com.google.android.material.textfield.TextInputLayout
        android:id="@+id/TIPercent"
style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropdownMenu"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="50dp"
```

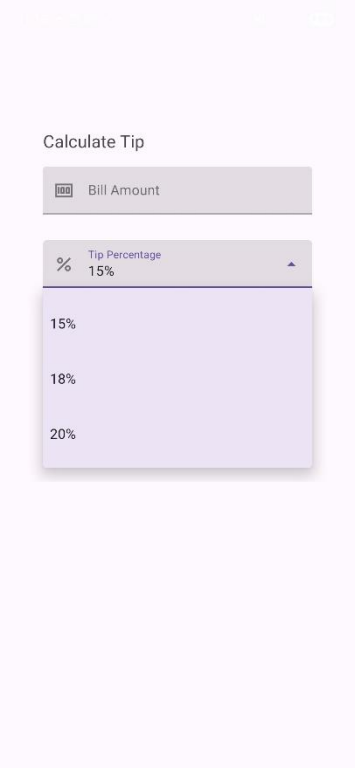
```
        android:layout_marginTop="30dp"
        android:layout_marginEnd="50dp"
        android:hint="Tip Percentage"
app:startIconDrawable="@drawable/baseline_percent_24"
app:layout_constraintTop_toBottomOf="@id/TIBill"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <AutoCompleteTextView
            android:id="@+id/ACTipPercent"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:paddingStart="50dp"
            android:inputType="none"/>


</com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:id="@+id/TVRound"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="50dp"
        android:text="Round up tip?"
        android:textSize="20sp"
app:layout_constraintBaseline_toBaselineOf="@id/SRound"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/TIPercent"/>


<com.google.android.material.materialswitch.MaterialSwitch
        android:id="@+id/SRound"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="40dp"
        android:layout_marginEnd="50dp"
        android:checked="false"
        app:layout_constraintEnd_toEndOf="parent"
```

```
app:layout_constraintTop_toBottomOf="@+id/TIPercent">


</com.google.android.material.materialswitch.MaterialSwitch>

    <TextView
        android:id="@+id/TVTipAmount"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="55dp"
        android:layout_marginTop="30dp"
        android:text="Tip Amount : $0.00"
        android:textSize="35sp"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/SRound" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

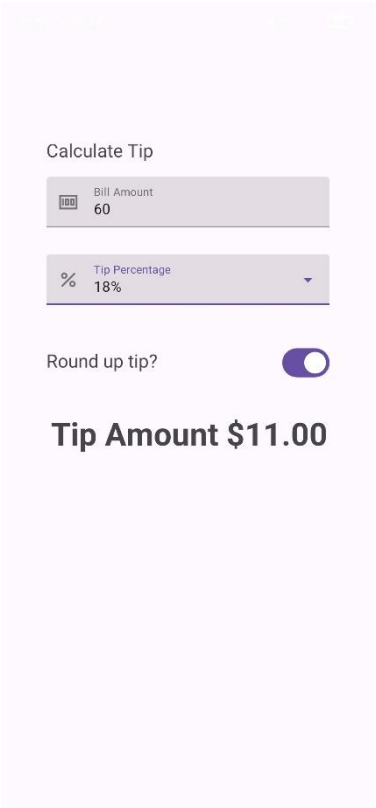
B. Output Program



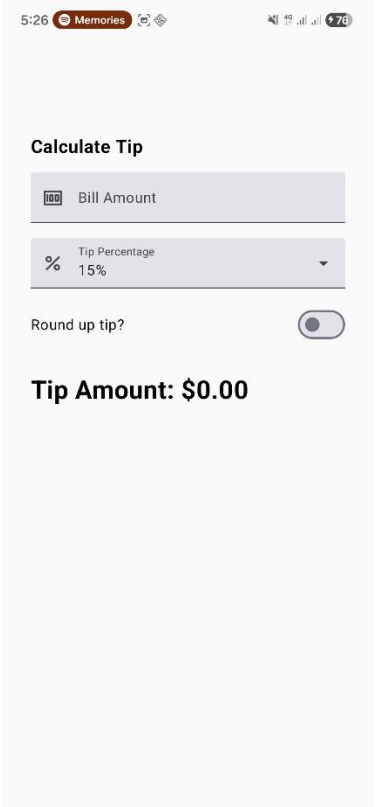
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal



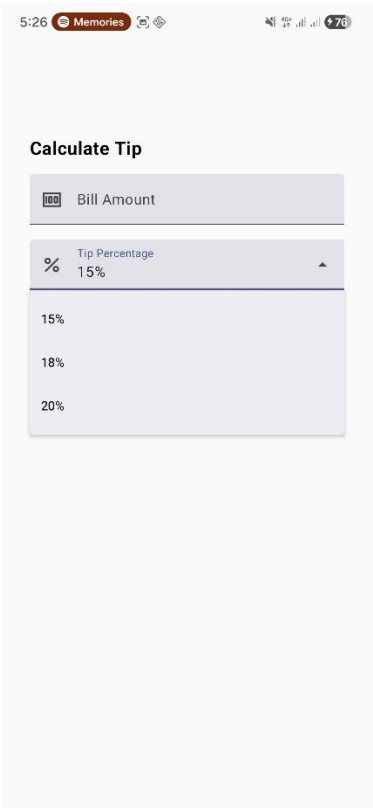
Gambar 5. Screenshot Hasil Jawaban Soal XML Persentase Tip



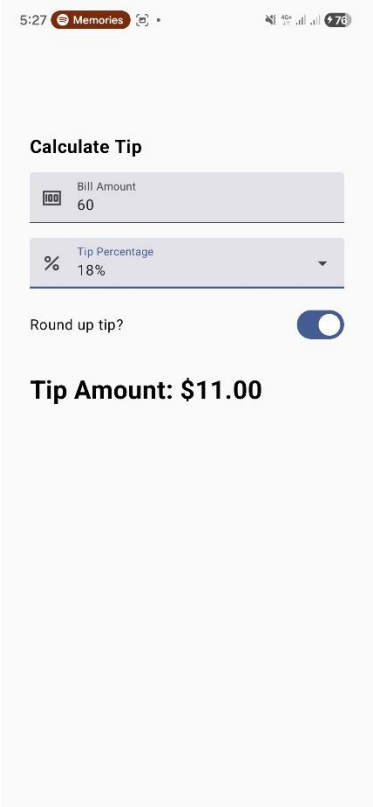
Gambar 6. Screenshot Hasil Jawaban Soal XML Setelah Dijalankan



Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal



Gambar 8. Screenshot Hasil Jawaban Soal Compose Persentase Tip



Gambar 9. Screenshot Hasil Jawaban Soal Compose Setelah Dijalankan

C. Pembahasan

XmlActivity.kt XML

Pada baris [1] terdapat ‘package com.example.xml’ yang mendefinisikan nama paket aplikasi Android ini, digunakan untuk mengorganisasikan kode dan komponen aplikasi.

Pada baris [3] sampai [7] terdapat beberapa directive import:

- Pada baris [3] terdapat ‘import android.os.Bundle’ yang digunakan untuk menyimpan state dan mengirim data antar Activity, digunakan pada ‘override fun onCreate(savedInstanceState: Bundle?)’.
- Pada baris [4] terdapat ‘import android.widget.ArrayAdapter’ yang berfungsi sebagai jembatan untuk mengubah data dari Array menjadi tampilan yang dapat dipahami oleh UI, khususnya untuk komponen dropdown.
- Pada baris [5] terdapat ‘import androidx.core.widget.addTextChangedListener’ yang merupakan listener untuk mendeteksi perubahan teks secara reaktif pada sebuah kolom input.
- Pada baris [6] terdapat ‘import androidx.appcompat.app.AppCompatActivity’ yang berfungsi sebagai cetakan untuk Activity yang mendukung fitur kompatibilitas, sehingga Activity dapat menggunakan fitur Android terbaru pada versi Android yang lebih lama.
- Pada baris [7] terdapat ‘import com.example.xml.databinding.ActivityMainBinding’ yang merupakan kelas binding yang di-generate otomatis oleh View Binding untuk menghubungkan kode Kotlin dengan komponen UI di ‘activity_main.xml’.

Pada baris [9] terdapat deklarasi ‘class MainActivity : AppCompatActivity()’ yang mendefinisikan class utama Activity dan mewarisi ‘AppCompatActivity’.

Pada baris [10] terdapat ‘private lateinit var binding: ActivityMainBinding’ yang mendeklarasikan variabel binding bertipe ‘ActivityMainBinding’. Kata kunci ‘lateinit’ menandakan bahwa variabel ini akan diinisialisasi nanti, bukan saat deklarasi.

Pada baris [12] sampai [15] terdapat fungsi ‘onCreate()’ yang merupakan lifecycle method utama Activity:

- Pada baris [13] terdapat ‘super.onCreate(savedInstanceState)’ yang memanggil implementasi dari kelas induk untuk menyiapkan infrastruktur Android.

- Pada baris [14] terdapat ‘binding = ActivityMainBinding.inflate(layoutInflater)’ yang menginisialisasi binding dengan meng-inflate layout ‘activity_main.xml’ menjadi objek yang dapat diakses di Kotlin.
- Pada baris [15] terdapat ‘setContentView(binding.root)’ yang menghubungkan Activity dengan layout yang sudah di-inflate melalui binding.

Pada baris [17] sampai [20] terdapat pengaturan komponen dropdown:

- Pada baris [17] terdapat ‘val tipOption = listOf(“15%”, “18%”, “20%”)’ yang mendefinisikan daftar pilihan persentase tip menggunakan ‘listOf’ yang bersifat immutable.
- Pada baris [18] terdapat ‘val adapter = ArrayAdapter(...)’ yang membuat adapter untuk menghubungkan data ‘tipOption’ ke tampilan dropdown dengan layout bawaan Android ‘simple_dropdown_item_1line’.
- Pada baris [19] terdapat ‘binding.ACTipPercent.setAdapter(adapter)’ yang memasang adapter ke dropdown agar pilihan tip tampil saat ditekan.
- Pada baris [20] terdapat ‘binding.ACTipPercent.setText(tipOption[0], false)’ yang mengatur nilai default dropdown menjadi “15%” (index 0) tanpa memfilter pilihan yang tersedia.

Pada baris [22] sampai [35] terdapat fungsi ‘calculateTip()’ yang bertugas menghitung dan menampilkan jumlah tip:

- Pada baris [23] terdapat ‘val billAmount = binding.TIEBill.text.toString().toDoubleOrNull() ?: 0.0’ yang mengambil teks dari kolom input bill dan mengkonversinya ke tipe Double. Jika input tidak valid atau kosong, nilainya akan menjadi 0.0.
- Pada baris [24] sampai [29] terdapat ekspresi ‘when’ yang menentukan nilai persentase tip berdasarkan pilihan dropdown. Jika “15%” dipilih maka nilainya 0.15, “18%” menjadi 0.18, “20%” menjadi 0.20, dan ‘else’ menangani kondisi tak terduga dengan nilai default 0.0.
- Pada baris [30] terdapat ‘var tipAmount = billAmount * tipPercentage’ yang menghitung jumlah tip berdasarkan nominal tagihan dan persentase yang dipilih.

- Pada baris [31] sampai [33] terdapat kondisi ‘if (binding.SRound.isChecked)’ yang memeriksa apakah switch pembulatan aktif. Jika ya, pada baris [32] nilai tip dibulatkan ke atas menggunakan ‘kotlin.math.ceil()’.
- Pada baris [34] terdapat ‘binding.TVTipAmount.text = “Tip Amount: ${“$”}${“%.2f”.format(tipAmount)}”’ yang menampilkan hasil perhitungan tip pada ‘TVTipAmount’ dengan format dua angka desimal.

Pada baris [37] sampai [39] terdapat tiga event listener yang masing-masing memanggil ‘calculateTip()’ secara otomatis:

- Pada baris [37] terdapat ‘binding.TIEBill.addTextChangedListener’ yang aktif setiap kali teks pada input bill berubah.
- Pada baris [38] terdapat ‘binding.ACTipPercent.setOnItemClickListener’ yang aktif saat pengguna memilih persentase dari dropdown.
- Pada baris [39] terdapat ‘binding.SRound.setOnCheckedChangeListener’ yang aktif saat kondisi switch berubah.

Pada baris [40] dan [41] terdapat penutup kurung kurawal yang mengakhiri fungsi ‘onCreate()’ dan class ‘MainActivity’.

2.3activity_main.xml

Pada baris [1] terdapat deklarasi ‘<?xml version=”1.0” encoding=”utf-8”?>’ yang merupakan header standar file XML, menunjukkan versi XML yang digunakan adalah 1.0 dengan encoding UTF-8.

Pada baris [2] sampai [9] terdapat tag pembuka elemen root ‘<androidx.constraintlayout.widget.ConstraintLayout>’ dengan atribut-atributnya:

- Pada baris [3] sampai [5] terdapat ‘xmlns:android’, ‘xmlns:app’, dan ‘xmlns:tools’ yang mendefinisikan namespace XML yang diperlukan agar atribut Android dapat dikenali.
- Pada baris [6] terdapat ‘android:id=”@+id/main”’ yang memberikan ID pada layout utama.
- Pada baris [7] dan [8] terdapat ‘android:layout_width=”match_parent”’ dan ‘android:layout_height=”match_parent”’ yang membuat layout mengisi seluruh lebar dan tinggi layar.

- Pada baris [9] terdapat ‘tools:context=”.MainActivity”’ yang menghubungkan layout ini dengan MainActivity untuk keperluan preview di Android Studio.

Pada baris [11] sampai [20] terdapat komponen ‘TextView’ pertama sebagai judul halaman:

- Pada baris [12] terdapat 'android:id=”@+id/TVCalculate”’ yang memberikan ID pada TextView ini.
- Pada baris [13] dan [14] terdapat ‘android:layout_width=”wrap_content”’ dan ‘android:layout_height=”wrap_content”’ yang membuat ukuran TextView menyesuaikan isi teksnya.
- Pada baris [15] dan [16] terdapat ‘android:layout_marginStart=”50dp”’ dan ‘android:layout_marginTop=”150dp”’ yang memberikan jarak dari kiri dan atas layar.
- Pada baris [17] terdapat ‘android:text=”Calculate Tip”’ yang menampilkan teks judul dan pada baris [18] terdapat ‘android:textSize=”20sp”’ yang mengatur ukuran teksnya.
- Pada baris [19] dan [20] terdapat ‘app:layout_constraintStart_toStartOf=”parent”’ dan ‘app:layout_constraintTop_toTopOf=”parent”’ yang mengaitkan posisi TextView ke sisi kiri dan atas dari parent.

Pada baris [22] sampai [42] terdapat komponen ‘TextInputLayout’ pertama sebagai wadah input nominal tagihan:

- Pada baris [23] terdapat ‘android:id=”@+id/TIBill”’ yang memberikan ID pada komponen ini.
- Pada baris [24] terdapat ‘style=”@style/Widget.MaterialComponents.TextInputLayout.FilledBox”’ yang menerapkan tampilan Material Design dengan background abu-abu tanpa border.
- Pada baris [25] terdapat ‘android:layout_width=”0dp”’ yang membuat lebar mengikuti constraint kiri dan kanan sehingga komponen memenuhi lebar yang tersedia.
- Pada baris [27], [28], dan [29] terdapat ‘android:layout_marginStart=”50dp”’, 'android:layout_marginTop=”16dp”’, dan ‘android:layout_marginEnd=”50dp”’ yang memberikan jarak dari tepi layar.

- Pada baris [30] terdapat ‘android:hint=”Bill Amount”’ yang menampilkan teks placeholder pada kolom input.
- Pada baris [31] terdapat ‘app:startIconDrawable=”@drawable/baseline_money_24”’ yang menambahkan ikon uang di sisi kiri kolom input.
- Pada baris [32], [33], dan [34] terdapat ‘app:layout_constraintTop_toBottomOf=”@id/TVCalculate”’, ‘app:layout_constraintStart_toStartOf=”parent”’, dan ‘app:layout_constraintEnd_toEndOf=”parent”’ yang mengatur posisi komponen ini tepat di bawah TVCalculate dan memenuhi lebar layar.
- Pada baris [36] sampai [41] terdapat ‘TextInputEditText’ di dalam ‘TextInputLayout’ dengan ‘android:id=”@+id/TIEBill”’ pada baris [37] sebagai ID yang direferensikan di Kotlin, ‘android:paddingStart=”50dp”’ pada baris [40] untuk menggeser teks input agar tidak bertumpuk dengan icon, dan ‘android:inputType=”numberDecimal”’ pada baris [41] agar keyboard angka desimal muncul saat input.

Pada baris [44] sampai [65] terdapat komponen ‘TextInputLayout’ kedua sebagai dropdown pemilihan persentase tip:

- Pada baris [45] terdapat ‘android:id=”@+id/TIPercent”’ yang memberikan ID pada komponen ini.
- Pada baris [46] terdapat ‘style=”@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropdownMenu”’ yang menerapkan gaya Material Design khusus untuk exposed dropdown menu yang otomatis menambahkan ikon panah di sisi kanan.
- Pada baris [49], [50], dan [51] terdapat ‘android:layout_marginStart=”50dp”’, ‘android:layout_marginTop=”30dp”’, dan ‘android:layout_marginEnd=”50dp”’ yang memberikan jarak dari tepi layar.
- Pada baris [52] terdapat ‘android:hint=”Tip Percentage”’ yang menampilkan teks placeholder pada dropdown.
- Pada baris [53] terdapat ‘app:startIconDrawable=”@drawable/baseline_percent_24”’ yang menambahkan ikon persen di sisi kiri.

- Pada baris [54], [55], dan [56] terdapat ‘app:layout_constraintTop_toBottomOf=”@id/TIBill”’, ‘app:layout_constraintStart_toStartOf=”parent”’, dan ‘app:layout_constraintEnd_toEndOf=”parent”’ yang memposisikan komponen ini tepat di bawah TIBill dan memenuhi lebar layar.
- Pada baris [58] sampai [63] terdapat ‘AutoCompleteTextView’ di dalam 'TextInputLayout' dengan ‘android:id=”@+id/ACTipPercent”’ pada baris [59] sebagai ID yang direferensikan di Kotlin, ‘android:paddingStart=”50dp”’ pada baris [62] untuk menggeser teks agar tidak bertumpuk dengan icon, dan ‘android:inputType=”none”’ pada baris [63] agar keyboard tidak muncul saat dropdown ditekan.

Pada baris [67] sampai [76] terdapat ‘TextView’ kedua sebagai label untuk switch pembulatan:

- Pada baris [68] terdapat ‘android:id=”@+id/TVRound”’ yang memberikan ID pada TextView ini.
- Pada baris [71] terdapat ‘android:layout_marginStart=”50dp”’ yang memberikan jarak dari sisi kiri layar.
- Pada baris [72] terdapat ‘android:text=”Round up tip?”’ dan pada baris [73] terdapat ‘android:textSize=”20sp”’ yang menampilkan teks keterangan switch dengan ukuran 20sp.
- Pada baris [74] terdapat ‘app:layout_constraintBaseline_toBaselineOf=”@id/SRound”’ yang meratakan baseline teks TVRound dengan SRound sehingga keduanya terlihat sejajar secara vertikal meskipun tingginya berbeda.
- Pada baris [76] terdapat ‘app:layout_constraintTop_toBottomOf=”@+id/TIPercent”’ yang memposisikan komponen ini di bawah TIPercent.

Pada baris [78] sampai [88] terdapat komponen ‘MaterialSwitch’ sebagai toggle pembulatan tip:

- Pada baris [79] terdapat ‘android:id=”@+id/SRound”’ yang memberikan ID yang direferensikan di Kotlin.

- Pada baris [82] dan [83] terdapat ‘android:layout_marginTop=”40dp”’ dan ‘android:layout_marginEnd=”50dp”’ yang memberikan jarak dari atas dan kanan layar.
- Pada baris [84] terdapat ‘android:checked=”false”’ yang mengatur kondisi awal switch dalam keadaan mati.
- Pada baris [85] terdapat ‘app:layout_constraintEnd_toEndOf=”parent”’ yang memposisikan switch ke sisi kanan layar dan pada baris [86] terdapat “app:layout_constraintTop_toBottomOf=”@+id/TIPercent”’ yang memposisikannya di bawah TIPercent.

Pada baris [90] sampai [100] terdapat ‘TextView’ terakhir yang menampilkan hasil perhitungan tip:

- Pada baris [91] terdapat ‘android:id=”@+id/TVTipAmount”’ sebagai ID yang direferensikan di Kotlin untuk memperbarui teks hasil kalkulasi.
- Pada baris [92] terdapat ‘android:layout_width=”0dp”’ dan pada baris [93] terdapat ‘android:layout_height=”wrap_content”’ yang mengatur ukuran TextView.
- Pada baris [94] dan [95] terdapat ‘android:layout_marginStart=”55dp”’ dan ‘android:layout_marginTop=”30dp”’ yang memberikan jarak dari kiri dan atas.
- Pada baris [96] terdapat ‘android:text=”Tip Amount : $0.00”’ sebagai nilai awal sebelum kalkulasi dilakukan.
- Pada baris [97] terdapat ‘android:textSize=”35sp”’ dan pada baris [98] terdapat ‘android:textStyle=”bold”’ yang membuat teks hasil tampil besar dan tebal.
- Pada baris [99] dan [100] terdapat ‘app:layout_constraintStart_toStartOf=”parent”’ dan ‘app:layout_constraintTop_toBottomOf=”@+id/SRound”’ yang memposisikan TextView ini di bawah SRound.

Pada baris [102] terdapat ‘</androidx.constraintlayout.widget.ConstraintLayout>’ yang menutup elemen root dan menandai akhir dari file layout XML.

MainActivity.kt Compose

Pada baris [1] terdapat ‘package com.example.composeModul2’ yang mendefinisikan nama paket aplikasi Compose ini.

Pada baris [3] sampai [21] terdapat beberapa directive import:

- Pada baris [3] terdapat ‘import android.os.Bundle’ yang digunakan untuk menyimpan state pada lifecycle Activity.
- Pada baris [4] terdapat ‘import androidx.activity.ComponentActivity’ yang merupakan kelas dasar Activity khusus untuk Jetpack Compose, berbeda dengan XML yang menggunakan ‘AppCompatActivity’.
- Pada baris [5] terdapat ‘import androidx.activity.compose.setContent’ yang digunakan untuk menghubungkan Activity dengan konten Composable.
- Pada baris [6] terdapat ‘import androidx.activity.enableEdgeToEdge’ yang mengaktifkan tampilan layar penuh hingga area system bar.
- Pada baris [7] terdapat ‘import androidx.compose.foundation.layout.*’ yang mengimpor semua komponen layout dasar Compose seperti ‘Column’, ‘Row’, dan ‘Spacer’.
- Pada baris [8] terdapat ‘import androidx.compose.foundation.text.KeyboardOptions’ yang digunakan untuk mengatur opsi keyboard pada input teks.
- Pada baris [9] terdapat ‘import androidx.compose.material3.*’ yang mengimpor semua komponen Material 3 seperti ‘TextField’, ‘Switch’, dan ‘ExposedDropdownMenuBox’.
- Pada baris [10] terdapat ‘import androidx.compose.runtime.*’ yang mengimpor fungsi-fungsi state management Compose seperti ‘remember’ dan ‘mutableStateOf’.
- Pada baris [11] sampai [16] terdapat import untuk ‘Alignment’, ‘Modifier’, ‘FontWeight’, ‘KeyboardType’, ‘dp’, dan ‘sp’ yang masing-masing digunakan untuk mengatur tampilan dan ukuran komponen UI.
- Pada baris [17] terdapat ‘import com.example.composeModul2.ui.theme.COMPOSETheme’ yang mengimpor tema aplikasi Compose.
- Pada baris [18] terdapat ‘import kotlin.math.ceil’ yang digunakan untuk membulatkan nilai tip ke atas.
- Pada baris [19] sampai [21] terdapat import untuk ‘Icons’, ‘Icons.Default.Money’, dan ‘Icons.Default.Percent’ yang digunakan sebagai ikon pada komponen input.

Pada baris [24] terdapat deklarasi ‘class MainActivity : ComponentActivity()’ yang mendefinisikan class utama Activity dan mewarisi ‘ComponentActivity’ khusus untuk Compose.

Pada baris [25] sampai [33] terdapat fungsi ‘onCreate()’ sebagai lifecycle method utama Activity:

- Pada baris [26] terdapat ‘super.onCreate(savedInstanceState)’ yang memanggil implementasi dari kelas induk.
- Pada baris [27] terdapat ‘enableEdgeToEdge()’ yang mengaktifkan tampilan layar penuh.
- Pada baris [28] sampai [32] terdapat ‘setContent’ yang merupakan cara Compose menghubungkan UI ke Activity. Di dalamnya terdapat ‘COMPOSETheme’ pada baris [29] sebagai pembungkus tema aplikasi, dan ‘TipCalculatorApp()’ pada baris [30] sebagai fungsi Composable utama yang dipanggil.

Pada baris [36] terdapat ‘@OptIn(ExperimentalMaterial3Api::class)’ yang diperlukan karena beberapa komponen Material 3 seperti ‘ExposedDropdownMenuBox’.

Pada baris [37] terdapat ‘@Composable’ yang menandai bahwa fungsi berikutnya adalah fungsi Composable yang dapat digunakan untuk membangun UI.

Pada baris [38] terdapat deklarasi ‘fun TipCalculatorApp()’ sebagai fungsi Composable utama yang berisi seluruh tampilan aplikasi.

Pada baris [39] sampai [42] terdapat deklarasi state menggunakan ‘remember’ dan ’mutableStateOf’:

- Pada baris [39] terdapat ‘var billAmount by remember { mutableStateOf(“”) }’ sebagai state untuk menyimpan nilai input tagihan, dengan nilai awal string kosong.
- Pada baris [40] terdapat ‘var selectedTip by remember { mutableStateOf(“15%”) }’ sebagai state untuk menyimpan pilihan persentase tip, dengan nilai default “15%”.
- Pada baris [41] terdapat ‘var roundUp by remember { mutableStateOf(false) }’ sebagai state untuk menyimpan kondisi switch pembulatan, dengan nilai default ‘false’.
- Pada baris [42] terdapat ‘var expanded by remember { mutableStateOf(false) }’ sebagai state untuk mengontrol apakah dropdown sedang terbuka atau tertutup.

Pada baris [44] terdapat ‘val tipOptions = listOf(“15%”, “18%”, “20%”)’ yang mendefinisikan daftar pilihan persentase tip yang bersifat immutable.

Pada baris [46] terdapat ‘val tipAmount = calculateTipAmount(billAmount, tipPercentage = selectedTip, roundUp)’ yang menghitung jumlah tip secara reaktif setiap kali state berubah.

Pada baris [48] sampai [52] terdapat komponen ‘Column’ sebagai layout utama yang menyusun elemen secara vertikal:

- Pada baris [50] terdapat ‘.fillMaxSize()’ yang membuat Column mengisi seluruh ukuran layar.
- Pada baris [51] terdapat ‘.padding(horizontal = 32.dp, vertical = 150.dp)’ yang memberikan jarak horizontal 32dp dan vertikal 150dp.

Pada baris [53] sampai [57] terdapat komponen ‘Text’ pertama sebagai judul halaman:

- Pada baris [54] terdapat ‘text = “Calculate Tip”’ yang menampilkan teks judul.
- Pada baris [55] terdapat ‘fontSize = 20.sp’ yang mengatur ukuran teks dan pada baris [56] terdapat ‘fontWeight = FontWeight.Bold’ yang membuat teks menjadi tebal.

Pada baris [59] terdapat ‘Spacer(modifier = Modifier.height(16.dp))’ yang memberikan jarak vertikal 16dp antar komponen. Spacer di Compose berfungsi sebagai pemisah antar elemen, menggantikan penggunaan margin di XML.

Pada baris [61] sampai [68] terdapat komponen ‘TextField’ pertama sebagai input nominal tagihan:

- Pada baris [62] terdapat ‘value = billAmount’ yang mengikat nilai input dengan state ‘billAmount’.
- Pada baris [63] terdapat ‘onValueChange = { billAmount = it }’ yang memperbarui state setiap kali teks berubah, sehingga UI otomatis ter-rekomposisi.
- Pada baris [64] terdapat ‘label = { Text(“Bill Amount”) }’ yang menampilkan label pada kolom input.
- Pada baris [65] terdapat ‘leadingIcon = { Icon(imageVector = Icons.Default.Money, contentDescription = null) }’ yang menambahkan ikon uang di sisi kiri input.
- Pada baris [66] terdapat ‘keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal)’ yang mengatur keyboard agar menampilkan angka desimal saat input.

- Pada baris [67] terdapat ‘modifier = Modifier.fillMaxWidth()’ yang membuat input memenuhi lebar layar.

Pada baris [70] terdapat ‘Spacer’ kedua dengan tinggi 16dp sebagai pemisah antara input bill dan dropdown.

Pada baris [72] sampai [101] terdapat komponen ‘ExposedDropdownMenuBox’ sebagai dropdown pemilihan persentase tip:

- Pada baris [73] terdapat ‘expanded = expanded’ yang mengikat kondisi dropdown dengan state ‘expanded’.
- Pada baris [74] terdapat ‘onExpandedChange = { expanded = !expanded }’ yang mengubah state ‘expanded’ setiap kali dropdown ditekan.
- Pada baris [76] sampai [86] terdapat ‘TextField’ di dalam ‘ExposedDropdownMenuBox’ dengan ‘value = selectedTip’ pada baris [77] yang menampilkan pilihan tip yang dipilih, ‘readOnly = true’ pada baris [79] agar input tidak bisa diketik manual, ‘leadingIcon’ pada baris [81] yang menambahkan ikon persen, ‘trailingIcon’ pada baris [82] yang menampilkan ikon panah dari ‘ExposedDropdownMenuDefaults.TrailingIcon’, dan ‘.menuAnchor()’ pada baris [85] yang menandai TextField sebagai titik anchor untuk dropdown.
- Pada baris [87] sampai [100] terdapat ‘ExposedDropdownMenu’ yang berisi daftar pilihan tip. Pada baris [91] sampai [99] terdapat ‘tipOptions.forEach’ yang melakukan iterasi pada setiap pilihan dan membuat ‘DropdownMenuItem’ untuk masing-masing opsi. Pada baris [95] terdapat ‘selectedTip = option’ dan pada baris [96] terdapat ‘expanded = false’ yang dipanggil saat item dipilih untuk memperbarui state dan menutup dropdown.

Pada baris [103] terdapat ‘Spacer’ ketiga dengan tinggi 16dp sebagai pemisah antara dropdown dan row switch.

Pada baris [105] sampai [115] terdapat komponen ‘Row’ yang menyusun label dan switch secara horizontal berdampingan:

- Pada baris [106] terdapat ‘modifier = Modifier.fillMaxWidth()’ agar Row memenuhi lebar layar.
- Pada baris [107] terdapat ‘verticalAlignment = Alignment.CenterVertically’ agar label dan switch rata secara vertikal.

- Pada baris [108] terdapat ‘horizontalArrangement = Arrangement.SpaceBetween’ agar label dan switch berada di ujung kiri dan kanan.
- Pada baris [110] terdapat ‘Text(text = “Round up tip?”, fontSize = 16.sp)’ yang menampilkan label keterangan switch.
- Pada baris [111] sampai [114] terdapat komponen ‘Switch’ dengan ‘checked = roundUp’ pada baris [112] yang mengikat kondisi switch dengan state ‘roundUp’, dan ‘onCheckedChange = { roundUp = it }’ pada baris [113] yang memperbarui state saat switch diubah.

Pada baris [117] terdapat ‘Spacer’ keempat dengan tinggi 32dp sebagai pemisah antara switch dan teks hasil.

Pada baris [119] sampai [123] terdapat komponen ‘Text’ terakhir yang menampilkan hasil perhitungan tip:

- Pada baris [120] terdapat ‘text = “Tip Amount: $$ {“%.2f”.format(tipAmount)}”’ yang menampilkan hasil tip dengan format dua angka desimal.
- Pada baris [121] terdapat ‘fontSize = 28.sp’ dan pada baris [122] terdapat ‘fontWeight = FontWeight.Bold’ yang membuat teks hasil tampil besar dan tebal.

Pada baris [127] sampai [138] terdapat fungsi ‘calculateTipAmount()’ yang bertugas menghitung jumlah tip:

- Pada baris [128] terdapat ‘val amount = billAmount.toDoubleOrNull() ?: 0.0’ yang mengkonversi input string menjadi Double, dengan nilai default 0.0 jika konversi gagal.
- Pada baris [129] sampai [134] terdapat ekspresi 'when' yang menentukan nilai persentase berdasarkan pilihan tip. ‘15%’ menghasilkan 0.15, ‘18%’ menghasilkan 0.18, ‘20%’ menghasilkan 0.20, dan ‘else’ menghasilkan nilai default 0.15.
- Pada baris [135] terdapat ‘var tip = amount * percentage’ yang menghitung jumlah tip.
- Pada baris [136] terdapat ‘if (roundUp) tip = ceil(tip)’ yang membulatkan nilai tip ke atas jika parameter ‘roundUp’ bernilai ‘true’.
- Pada baris [137] terdapat ‘return tip’ yang mengembalikan hasil perhitungan tip.

Pada baris [138] terdapat penutup kurung kurawal yang mengakhiri fungsi ‘calculateTipAmount()’.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan Implementasi XML dan Jetpack Compose

1. Cara Membangun UI

Pada implementasi XML, tampilan antarmuka dibangun menggunakan file layout terpisah yaitu 'activity_main.xml'. di dalamnya terdapat komponen UI seperti 'TextView', 'TextInputLayout', dan 'MaterialSwitch' dideklarasikan dalam file XML, kemudian dihubungkan ke logika Kotlin melalui View Binding. Hal ini berarti terdapat dua file yang harus dikelola secara bersamaan yaitu file XML untuk tampilan dan file Kotlin untuk logika.

Pada implementasi Jetpack Compose, seluruh tampilan dan logika dibangun dalam satu file Kotlin menggunakan fungsi-fungsi dari '@Composable'. Tidak ada file XML terpisah karena UI dideklarasikan langsung dalam kode Kotlin menggunakan komponen seperti 'Column', 'TextField', dan 'Switch'.

2. Manajemen State

Pada implementasi XML, perubahan UI dilakukan secara manual dengan memanggil event listener seperti 'addTextChangedListener', 'setOnItemClickListener', dan 'setOnCheckedChangeListener'. Setiap perubahan state harus secara eksplisit memperbarui komponen UI melalui 'binding.TVTipAmount.text'.

Pada implementasi Jetpack Compose, state dikelola menggunakan 'remember' dan 'mutableStateOf'. Ketika state berubah, Compose secara otomatis melakukan rekomposisi sehingga UI diperbarui tanpa perlu memanggil listener secara manual. Contohnya, perubahan pada 'billAmount' langsung memicu perhitungan ulang 'tipAmount' dan memperbarui tampilan secara otomatis.

3. Struktur Kode

Pada implementasi XML, struktur kode terbagi menjadi dua bagian yaitu deklarasi komponen di XML dan pengikatan logika di Kotlin. Posisi dan ukuran komponen ditentukan melalui atribut seperti 'app:layout_constraintTop_toBottomOf' dan 'android:layout_marginStart'.

Pada implementasi Jetpack Compose, seluruh struktur UI ditulis dalam satu fungsi Composable menggunakan pendekatan deklaratif. Tata letak diatur menggunakan 'Modifier' seperti '.fillMaxWidth()', '.padding()', dan '.fillMaxSize()' yang dirangkai secara berantai.

Kelebihan dan Kekurangan XML

1. Kelebihan XML:

Pertama, XML mudah dipahami oleh pemula karena struktur layoutnya terpisah dari logika kode, sehingga lebih mudah dibaca dan dipelajari. Pemisahan antara tampilan dan logika ini juga membuat kode lebih terorganisir bagi developer yang baru belajar Android.

Kedua, XML memiliki fitur preview langsung di Android Studio yang memungkinkan developer melihat tampilan UI secara real-time tanpa harus menjalankan aplikasi di emulator atau perangkat fisik. Hal ini mempercepat proses desain dan penyesuaian tampilan.

Ketiga, XML memiliki dokumentasi dan komunitas yang sangat luas karena sudah digunakan sejak awal pengembangan Android. Banyak tutorial, forum, dan referensi yang tersedia sehingga memudahkan developer dalam mencari solusi ketika menghadapi masalah.

Keempat, XML kompatibel dengan versi Android yang lebih lama sehingga aplikasi yang dibangun dengan XML dapat menjangkau lebih banyak pengguna tanpa khawatir masalah kompatibilitas perangkat.

2. Kekurangan XML:

Pertama, XML membutuhkan pengelolaan dua file sekaligus yaitu file layout XML dan file Kotlin, sehingga ketika terjadi perubahan pada UI, developer harus memastikan kedua file tersebut tetap sinkron satu sama lain.

Kedua, kode XML cenderung lebih verbose atau panjang karena setiap komponen membutuhkan banyak atribut untuk mengatur tampilan, posisi, dan ukurannya. Hal ini membuat file layout menjadi panjang dan kadang sulit dibaca terutama untuk tampilan yang kompleks.

Ketiga, manajemen state pada XML dilakukan secara manual menggunakan berbagai event listener seperti 'addTextChangedListener', 'setOnItemClickListener', dan 'setOnCheckedChangeListener'. Setiap perubahan UI harus dipanggil secara eksplisit, sehingga semakin kompleks aplikasi maka semakin banyak listener yang harus dikelola dan berpotensi menimbulkan bug.

Keempat, pengikatan antara komponen XML dan kode Kotlin melalui View Binding atau 'findViewById' menambah boilerplate code yang harus ditulis, meskipun View Binding sudah cukup membantu dibandingkan cara lama menggunakan 'findViewById'.

Kelebihan dan Kekurangan Jetpack Compose

1. Kelebihan Jetpack Compose:

Pertama, Jetpack Compose menggunakan pendekatan deklaratif dimana developer cukup mendeskripsikan bagaimana tampilan UI seharusnya berdasarkan state yang ada, dan Compose yang akan mengurus pembaruan UI secara otomatis. Hal ini membuat kode lebih mudah diprediksi dan lebih sedikit kemungkinan terjadinya bug terkait pembaruan UI.

Kedua, manajemen state pada Compose jauh lebih mudah menggunakan 'remember' dan 'mutableStateOf'. Ketika state berubah, Compose secara otomatis melakukan rekomposisi pada komponen yang terpengaruh tanpa perlu memanggil listener secara manual, sehingga kode menjadi lebih bersih dan ringkas.

Ketiga, Compose memungkinkan developer membangun seluruh UI hanya dalam satu file Kotlin tanpa perlu file XML terpisah. Hal ini membuat struktur proyek lebih sederhana dan mengurangi konteks perpindahan antara dua file yang berbeda saat mengembangkan fitur.

Keempat, komponen UI pada Compose sangat mudah digunakan kembali karena setiap fungsi '@Composable' dapat dipanggil di mana saja. Developer dapat membangun

komponen kustom yang modular dan reusable dengan cara yang lebih alami dibandingkan XML.

2. Kekurangan Jetpack Compose:

Pertama, Jetpack Compose memiliki kurva belajar yang lebih tinggi terutama bagi developer yang sudah terbiasa dengan pendekatan imperatif menggunakan XML. Konsep seperti rekomposisi, state hoisting, dan 'remember' membutuhkan waktu untuk dipahami dengan baik.

Kedua, beberapa komponen Material 3 pada Compose masih percobaan, seperti 'ExposedDropdownMenuBox' yang membutuhkan anotasi '@OptIn(ExperimentalMaterial3Api::class)'. Hal ini berarti API tersebut masih dapat berubah di versi mendatang dan berpotensi menyebabkan masalah kompatibilitas.

Ketiga, Compose membutuhkan versi minimum Android yang lebih tinggi dan versi library yang lebih baru dibandingkan XML. Hal ini dapat membatasi jangkauan aplikasi ke pengguna dengan perangkat Android yang lebih lama.

Keempat, performa Compose pada aplikasi yang sangat kompleks dengan banyak rekomposisi dapat menjadi masalah jika tidak dikelola dengan baik. Developer perlu memahami cara mengoptimalkan rekomposisi menggunakan teknik seperti 'derivedStateOf' dan 'key' agar aplikasi tetap berjalan dengan lancar.

SOAL ESSAY

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
2. Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Pembahasan

1. Apa saja fitur yang menjadi daya jual pada Kotlin?

Kotlin adalah bahasa pemrograman modern yang dikembangkan oleh JetBrains pada tahun 2011 dan telah diakui oleh Google sebagai bahasa resmi untuk pengembangan Android sejak 2017. Kotlin dirancang untuk menjadi lebih ringkas, ekspresif, dan aman dibandingkan Java.

Kotlin is a relatively new, fast-growing programming language known for its brevity, safety, and interoperability with Java. Kotlin is a statically typed, object-oriented programming language that is designed to interoperate fully with Java and run on the Java Virtual Machine (JVM) along with other platforms [1].

Kalimat di atas menjelaskan bahwa Kotlin merupakan bahasa pemrograman yang relatif baru namun sangat berkembang pesat, dikenal karena keringkasannya, keamanannya, dan kemampuannya untuk berjalan berdampingan dengan Java. Berikut adalah fitur-fitur unggulan yang menjadi daya jual Kotlin:

1. Null Safety

Null Safety is a Kotlin feature designed to significantly reduce the risk of null references, also known as The Billion-Dollar Mistake. Kotlin explicitly supports nullability as part of its type system, meaning you can explicitly declare which variables or properties are allowed to be null [1].

Kalimat di atas menjelaskan bahwa null safety merupakan fitur unggulan Kotlin yang dirancang untuk mencegah terjadinya null reference error. Dengan sistem tipe yang membedakan antara variabel nullable dan non-nullable, Kotlin memastikan potensi error akibat nilai null dapat terdeteksi sejak mulai dikompilasi. Developer dapat menggunakan operator '?', '?.', dan '?:' yaitu Elvis operator untuk menangani nilai null secara aman tanpa risiko aplikasi crash akibat NullPointerException.

2. Extension Functions

One of the most useful features related to nullable types is Kotlin's capacity to use extension functions. These functions make your code cleaner by allowing specific operations on nullable variables instead of constantly performing checks [1].

Dari kalimat di atas dapat dipahami bahwa extension functions memungkinkan developer menambahkan fungsi baru ke sebuah kelas yang sudah ada tanpa harus mewarisi atau memodifikasi kelas tersebut. Hal ini membuat kode lebih bersih dan mudah dibaca karena fungsi tambahan dapat dipanggil seolah-olah merupakan bagian dari kelas aslinya.

3. Coroutines

Kotlin features coroutines for simplified asynchronous programming, making it easier to handle operations like network requests and database queries without blocking the main thread [2].

Kalimat di atas menjelaskan bahwa coroutines merupakan fitur bawaan Kotlin yang memudahkan penanganan operasi asinkron. Coroutines memungkinkan penulisan kode asinkron dengan cara yang lebih sederhana dan mudah dibaca dibandingkan callback atau thread manual pada Java, sehingga aplikasi dapat berjalan lebih efisien tanpa risiko blocking pada main thread.

4. Smart Cast

Kotlin features smart casts that are not available with Java for Android development [2].

Dari kalimat di atas dapat dipahami bahwa smart cast merupakan fitur khusus Kotlin yang tidak dimiliki Java. Kotlin secara otomatis melakukan cast tipe variabel setelah dilakukan pengecekan tipe menggunakan ‘is’, sehingga developer tidak perlu melakukan cast secara manual karena compiler Kotlin sudah cukup cerdas untuk mengenali tipe variabel setelah pengecekan dilakukan.

5. Higher-Order Functions dan Lambda

Kotlin supports lambda expressions, higher-order functions, and more as part of its modern functional features [2].

Kalimat di atas menjelaskan bahwa Kotlin mendukung higher-order functions yang memungkinkan fungsi menerima fungsi lain sebagai parameter atau mengembalikan fungsi sebagai nilai. Fitur ini membuat kode lebih ringkas dan ekspresif terutama saat bekerja dengan koleksi data menggunakan fungsi seperti 'map', 'filter', dan 'forEach'.

6. Data Class

Kotlin introduces data classes which are classes primarily used to hold data, reducing the need for boilerplate code that would otherwise be required in Java [2].

Dari kalimat di atas dapat dipahami bahwa data class merupakan fitur Kotlin yang secara otomatis men-generate fungsi-fungsi boilerplate seperti 'equals()', 'hashCode()', 'toString()', dan 'copy()'. Hal ini sangat berguna untuk membuat kelas yang berfungsi sebagai wadah data tanpa harus menulis banyak kode berulang seperti di Java.

7. Concise Syntax

Some benefits of Kotlin are that semicolons are optional as a statement terminator in most cases, a newline is sufficient for the compiler to deduce that the statement has ended [1].

Kalimat di atas menjelaskan bahwa Kotlin memiliki sintaks yang jauh lebih singkat dibandingkan Java. Fitur seperti type inference, named parameters, default parameter values, dan string templates membuat kode Kotlin lebih pendek dan mudah dibaca tanpa mengorbankan keterbacaan kode.

8. Interoperability dengan Java

Kotlin is 100% interoperable with Java, meaning you can use Kotlin and Java code together in the same project, facilitating a smooth transition for existing Java projects [2].

Dari kalimat di atas dapat dipahami bahwa Kotlin sepenuhnya interoperable dengan Java. Developer dapat menggunakan library Java yang sudah ada di dalam proyek Kotlin dan sebaliknya, sehingga memudahkan migrasi proyek dari Java ke Kotlin secara bertahap tanpa harus menulis ulang seluruh kode dari awal.

Menurut saya fitur-fitur unggulan yang dimiliki Kotlin menjadikannya bahasa yang jauh lebih produktif dan aman untuk digunakan dalam pengembangan Android. Terutama fitur null safety yang secara langsung mencegah aplikasi crash akibat NullPointerException.

2. Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin

Null Safety

Null safety adalah fitur unggulan Kotlin yang mencegah terjadinya NullPointerException dengan membedakan variabel yang dapat bernilai null dan yang tidak sejak tahap kompilasi. Berikut adalah contoh penggunaan null safety pada kasus sederhana pencarian data mahasiswa berdasarkan NIM:

```
fun findStudent(nim: String): String? {
    val students = mapOf(
        "1234567" to "Lala",
        "7654321" to "Lili",
        "1010101" to "Lulu"
    )
    return students[nim]
}

fun main() {
    val nim1 = "1234567"
    val nim2 = "6767676"

    val student1 = findStudent(nim1)
    val student2 = findStudent(nim2)

    println(student1?.uppercase() ?: "Mahasiswa tidak ditemukan")
    println(student2?.uppercase() ?: "Mahasiswa tidak ditemukan")
}
```

Output :

```
LALA
Mahasiswa tidak ditemukan
```

Fungsi ‘findStudent()’ memiliki return type ‘String?’ yang berarti fungsi ini dapat mengembalikan nilai ‘String’ atau ‘null’. Ketika NIM ‘1234567’ dicari, fungsi mengembalikan ’”Lala”’, sedangkan ketika NIM ‘6767676’ dicari, fungsi mengembalikan ‘null’ karena NIM tersebut tidak ada di dalam data.

Pada baris ‘student1?.uppercase()’, operator ‘?.’ (safe call) digunakan untuk memanggil fungsi ‘uppercase()’ secara aman. Jika nilai ‘student1’ tidak null maka ‘uppercase()’ akan dieksekusi, sedangkan jika null maka ekspresi akan menghasilkan null tanpa menyebabkan crash.

Operator ‘?:’ (Elvis operator) pada bagian ‘?: “Mahasiswa tidak ditemukan”’ berfungsi sebagai nilai default ketika hasil ekspresi sebelumnya bernilai null. Sehingga ketika ‘student2’ bernilai null, program tidak crash melainkan menampilkan pesan ‘”Mahasiswa tidak ditemukan”’.

Tanpa fitur null safety seperti di Java, kode ini berpotensi menyebabkan NullPointerException ketika mencoba memanggil ‘uppercase()’ pada variabel yang bernilai null, yang dapat menyebabkan aplikasi berhenti secara tiba-tiba.

TAUTAN GITHUB

Link GitHub Modul 2

DAFTAR PUSTAKA

[1] S. S. Shinde and P. Adkar, "A Review Paper on Kotlin Programming Language," *International Journal of Trend in Scientific Research and Development,* vol. 5, no. 4, 2021.

[2] International Journal of Computer Applications Technology and Research, "Kotlin Programming Language: A Comprehensive Overview of Definition, Applications, Advantages, and Limitations," *International Journal of Computer Applications Technology and Research,* vol. 13, no. 3, pp. 07-09, 2024.